Appendix A. Code implementation of G3CN

Python code

```python
class OneAdjG3CN(nn.Module):
    def __init__(self, args):
        super(OneAdjG3CN, self).__init__()
        self.args = args
        self.W = Parameter(torch.Tensor(args.node_num, args.node_num))
        self.b = Parameter(torch.Tensor(args.node_num, 1))
        self.v = Parameter(torch.Tensor(args.node_num))
        self.reset_parameters()
    def reset_parameters(self):
        torch.nn.init.kaiming_normal_(self.W, nonlinearity='leaky_relu')
        torch.nn.init.normal_(self.b)
        torch.nn.init.normal_(self.v)
    def forward(self, X, A):
        fixed = torch.zeros_like(self.W, requires_grad=False)
        W = A * self.W + (1 - A) * fixed
        H = torch.matmul(W, X) + self.b
        if self.args.residual_alpha != 0:
            H = (1-self.args.residual_alpha) * torch.matmul(W, X) + self
                .args.residual_alpha * X + self.b
        H = F.leaky_relu(H, negative_slope=self.args.LeakyReLU_slope)
        v = self.v.unsqueeze(0).unsqueeze(2)
        H = v * H
        return H

class G3CN(nn.Module):
    def __init__(self, args, K=None):
        super(G3CN, self).__init__()
        self.args = args
        self.K = args.K
        if K is not None:
            self.K = K
        self.MAdjG3CNlist = nn.ModuleList([OneAdjG3CN(args) for _ in
            range(self.K)])
```

```
        # last layer for StandardScaler()
        self.last_linear_w = Parameter(torch.Tensor(args.node_num))
        self.last_linear_b = Parameter(torch.Tensor(args.node_num))
        self.reset_parameters()
    def reset_parameters(self):
        torch.nn.init.normal_(self.last_linear_w, mean=1.0, std=0.1)
        torch.nn.init.normal_(self.last_linear_b)
    def forward(self, X, A):
        """
        :param X: (batch_size, node_num, lag)
        :param A: (node_num, node_num)
        :return: H: (batch_size, node_num, lag)
        """
        H = [self.MAdjG3CNlist[i](X, A) for i in range(self.K)]
        H = torch.stack(H, dim=0).sum(dim=0)
        H = self.last_linear_w.unsqueeze(0).unsqueeze(2) * H + self.
            last_linear_b.unsqueeze(0).unsqueeze(2)
        return H

class Multi_Layer_G3CN(nn.Module):
    def __init__(self, args):
        super(Multi_Layer_G3CN, self).__init__()
        self.args = args
        G3CN_Layer_list = []
        layer_num = len(args.G3CN_Layer_K_nums)
        for i in range(layer_num):
            G3CN_Layer_list.append(G3CN(args, args.G3CN_Layer_K_nums[i])
                )
        self.G3CN_Layer_list = nn.ModuleList(G3CN_Layer_list)
    def forward(self, X, A):
        """
        :param X: (batch_size, node_num, lag)
        :param A: (node_num, node_num)
        :return: H: (batch_size, node_num, lag)
        """
        H = X
```

```
        for G3CN_Layer in self.G3CN_Layer_list:
            H = G3CN_Layer(H, A) + H * self.args.G3CN_Layer_residual
        return H
```

The above code implementation is applicable to tasks related to time series data. If it is applied to other fields, an additional step of node feature encoding needs to be supplemented.

Appendix B. The application for dynamic graph

In most time series tasks within industrial applications, static graphs are sufficient. Whether it is necessary to expend significant computational resources on dynamic graphs depends on whether the underlying physical interaction functions change, since the graph structure reflects the physical topology among sensors. [1] Nevertheless, certain complex industrial systems undergo significant structural alterations during operation, leading to changes in physical-level interactions or control laws, necessitating dynamic graphs. This section outlines brief extension strategies for G3CN:

(1) Snapshotting: store some observational data as snapshot, updating upon new sample arrival, and recalculate the adjacency matrix at fixed iteration intervals.

(2) Evolutionary Modeling: model the adjacency matrix's weight matrix as a hidden state or output of an RNN for dynamic evolution, following approaches like EvolveGCN [2]

(3) Incremental Propagation Updates: perform incremental updates along graph propagation paths during each iteration, avoiding full-graph recomputation, such as DyGCN [3].

Appendix C. Wavelet Packet Decomposition (WPD)

Wavelet packet decomposition is an effective tool for dealing with stationary and non-stationary time series. Its fast algorithm is called wavelets multi-resolution analysis, which is fast and easy for application , and has

[1] Specifically, in the majority of industrial equipment, only the input and output variables x and y vary, while the underlying physical interaction law f(x) remains unchanged. As long as the hardware topology of the circuit is fixed, the underlying physical interaction function does not change. In such cases, a static graph neural network essentially aims to approximate this function, even if it is piecewise due to varying operating conditions.

[2]* A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. Schardl, C. Leiserson, Evolvegcn: Evolving graph convolutional networks for dynamic graphs, Proceedings of the AAAI Conference on Artificial Intelligence 34 (04) (2020) 5363-5370. doi:10.1609/aaai.v34i04.5984. URL https://ojs.aaai.org/index.php/AAAI/article/view/5984

[3]* Z. Cui, Z. Li, S. Wu, X. Zhang, Q. Liu, L. Wang, M. Ai, Dygcn: Efficient dynamic graph embedding with graph convolutional network, IEEE Transactions on Neural Networks and Learning Systems 35 (4) (2024) 4635-4646. doi:10.1109/TNNLS.2022.3185527.

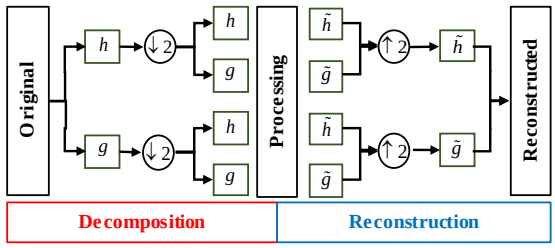

Fig. C.11. WPD tree

been widely applied as a powerful technique for compression, denoising, feature extraction, simplification, and modeling of time series in different fields of research, such as time series forecasting, pattern recognition. Its key technology Discrete wavelet transform (DWT) can be formulated as follows:

$$\mathrm{DWT}_x^{\psi}(\tau, s) = \frac{1}{\sqrt{s_0^m}} \int_{-\infty}^{+\infty} x(t)\, \psi^*\left(\frac{t - n\tau_0 s_0^m}{s_0^m}\right) dt, \tag{C.1}$$

where x denotes the input variables. τ and s are the transition and scaling parameters, respectively. ψ is a window function, which is so-called a mother wavelet. s_0 and τ_0 are greater than 1. The parameters m and n are integers. In our work, WPD is employed as a preprocessing step to decompose time series into various sub-components corresponding to different frequency bands.

Appendix D. Maximal Information Coefficient (MIC)

There are several parametric and non-parametric measures for correlation, such as Pearson correlation coefficient, cosine similarity, etc. However,these metrics are limited to the kind of correlations they can detect, or they make assumptions about the underlying distributions of the variables. For example, Pearson correlation coefficient only detects linear correlations, two variables may have a Pearson = 0.0 but still be correlated. The maximal information coefficient (MIC) breaks this constraint. In statistics, the MIC is a measure of the strength of the linear or nonlinear correlations between two variables. For two series M and N, we define them as $D = \{(m_i, n_i), i = 1, 2, \cdots, T\}$, the T is the length of these series. The calculation of MIC is on the basis of mutual information MI, which is given by:

$$\mathrm{MI}(M, N) = \sum_{M,N} p(m, n) \log \frac{p(m, n)}{p(m)p(n)}, \tag{D.1}$$

where the $p(m, n)$ is the joint probability density. $p(m)$ and $p(n)$ are the marginal distributions of $p(m, n)$. Finally, the MIC values can be obtained by:

$$\mathrm{MIC}(M, N) = \max_{|M||N|<B(T)} \frac{\mathrm{MI}(M; N)}{\log \min \{|M|, |N|\}}, \tag{D.2}$$

The grid size was limited with $|M| \times |N| \leq B(T)$ ($B(T) = T^a$) to reduce the calculation work. In general, MIC works well in practice when $B(T) = T^{0.6}$. The interval of the MIC value was [0, 1]. The closer the value was to 1, the stronger the correlation between the two variables was.

Appendix E. Comparison of MIC and other nonlinear correlation measures

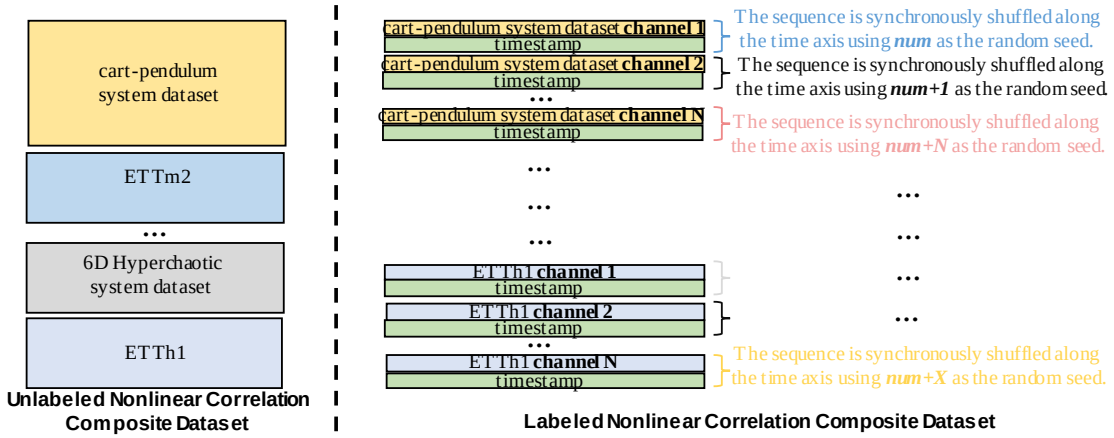

Fig. E.12. The construction of Unlabeled Nonlinear Correlation Composite Dataset and Labeled Nonlinear Correlation Composite Dataset.

To validate the superiority of MIC in capturing nonlinear correlations, comparative experiments were conducted in this section. All datasets utilized in Section 6 and Section 7 were concatenated along the sensor dimension to construct a composite dataset encompassing diverse forms of nonlinear correlations. Although this dataset contains a rich variety and quantity of nonlinear correlations, we do not have the true correlation labels for this dataset. This dataset can only provide qualitative visualizations, demonstrating the impact of various graph computation thresholds on graph sparsity under different nonlinear measures, offering overall qualitative references. Thus, we supplement a dataset with sparser correlations but possessing true correlation labels (still including various types of nonlinear correlations, but sparser) for quantitative

comparative experiments, calculating metrics such as Accuracy, F1-score, and AUC-ROC between the graph structures computed under various nonlinear measures and the true correlation labels. The former dataset is named UNCC Dataset (Unlabeled Nonlinear Correlation Composite Dataset), while the latter is named LNCC Dataset (Labeled Nonlinear Correlation Composite Dataset). Their construction methods are illustrated in Figure E.12.

Appendix E.1. Results on Labeled Nonlinear Correlation Composite Dataset

Graph calculate threshold	MIC	Mutual information	Copent	Cosine	Kendall
0.000	0.017	0.017	0.017	0.017	0.017
0.050	1.000	1.000	0.954	0.998	0.998
0.100	1.000	0.998	0.962	0.998	0.996
0.150	1.000	0.997	0.960	0.997	0.996
0.200	1.000	0.996	0.972	0.996	0.995
0.250	0.999	0.996	0.971	0.996	0.995
0.300	0.999	0.995	0.984	0.995	0.994
0.350	0.999	0.995	0.983	0.995	0.994
0.400	0.999	0.994	0.983	0.995	0.994
0.450	0.998	0.994	0.982	0.995	0.993
0.500	0.998	0.994	0.982	0.994	0.993
0.550	0.998	0.994	0.994	0.994	0.993
0.600	0.997	0.993	0.994	0.994	0.993
0.650	0.997	0.993	0.993	0.993	0.993
0.700	0.996	0.993	0.993	0.993	0.993
0.750	0.996	0.993	0.993	0.993	0.993
0.800	0.996	0.993	0.993	0.993	0.993
0.850	0.995	0.992	0.993	0.993	0.993
0.900	0.994	0.992	0.992	0.993	0.993
0.950	0.994	0.992	0.992	0.993	0.993
1.000	0.992	0.992	0.992	0.992	0.992

(a) Accuracy

Graph calculate threshold	MIC	Mutual information	Copent	Cosine	Kendall
0.000	0.033	0.033	0.033	0.033	0.033
0.050	1.000	0.996	0.419	0.939	0.929
0.100	1.000	0.943	0.459	0.919	0.868
0.150	0.996	0.894	0.419	0.899	0.857
0.200	0.992	0.873	0.494	0.857	0.835
0.250	0.983	0.863	0.479	0.852	0.818
0.300	0.974	0.835	0.612	0.829	0.800
0.350	0.974	0.818	0.589	0.824	0.788
0.400	0.970	0.800	0.574	0.818	0.782
0.450	0.947	0.788	0.565	0.806	0.756
0.500	0.929	0.769	0.550	0.776	0.750
0.550	0.919	0.763	0.782	0.769	0.743
0.600	0.894	0.743	0.769	0.769	0.737
0.650	0.889	0.737	0.756	0.750	0.730
0.700	0.868	0.730	0.750	0.743	0.730
0.750	0.863	0.723	0.730	0.743	0.723
0.800	0.857	0.717	0.723	0.743	0.717
0.850	0.824	0.696	0.717	0.730	0.717
0.900	0.776	0.689	0.703	0.717	0.717
0.950	0.769	0.681	0.696	0.710	0.717
1.000	0.703	0.674	0.667	0.667	0.696

(b) F1-score

Graph calculate threshold	MIC	Mutual information	Copent	Cosine	Kendall
0.000	0.500	0.500	0.500	0.500	0.500
0.050	1.000	0.996	0.976	0.946	0.933
0.100	1.000	0.946	0.968	0.925	0.883
0.150	0.996	0.904	0.914	0.908	0.875
0.200	0.992	0.888	0.900	0.875	0.858
0.250	0.983	0.879	0.883	0.871	0.846
0.300	0.975	0.858	0.881	0.854	0.833
0.350	0.975	0.846	0.860	0.850	0.825
0.400	0.971	0.833	0.848	0.846	0.821
0.450	0.950	0.825	0.839	0.838	0.804
0.500	0.933	0.813	0.827	0.817	0.800
0.550	0.925	0.808	0.821	0.813	0.796
0.600	0.904	0.796	0.813	0.813	0.792
0.650	0.900	0.792	0.804	0.800	0.788
0.700	0.883	0.788	0.800	0.796	0.788
0.750	0.879	0.783	0.788	0.796	0.783
0.800	0.875	0.779	0.783	0.796	0.779
0.850	0.850	0.767	0.779	0.788	0.779
0.900	0.817	0.763	0.771	0.779	0.779
0.950	0.813	0.758	0.767	0.775	0.779
1.000	0.771	0.754	0.750	0.750	0.767

(c) ROC-AUC

Fig. E.13. Quantitative comparison results of different nonlinear correlation measures on the LNCC Dataset.

On the LNCC Dataset, we calculated metrics such as Accuracy, F1-score, and AUC-ROC between the graph structures computed under various nonlinear measures and the true correlation labels, with results presented in Figure E.13. As the weight threshold increases from 0 to 1, the sparsity of the constructed graph structure gradually increases. In the comparison with the ground-truth labels, better results are indicated in red, whereas poorer results are shown in blue. It can be observed that MIC achieves the best overall performance, occupying most of the red regions in the three representative metrics: Accuracy, F1-score, and AUC-ROC.

Appendix E.2. Results on Unlabeled Nonlinear Correlation Composite Dataset

Meanwhile, to provide an overall qualitative reference, we also compared the graph sparsity under various nonlinear correlation measures on the UNCC Dataset, with results shown in Figure E.14. As illustrated, MIC demonstrated superior overall performance. In comparison to Kendall coefficient, mutual information and

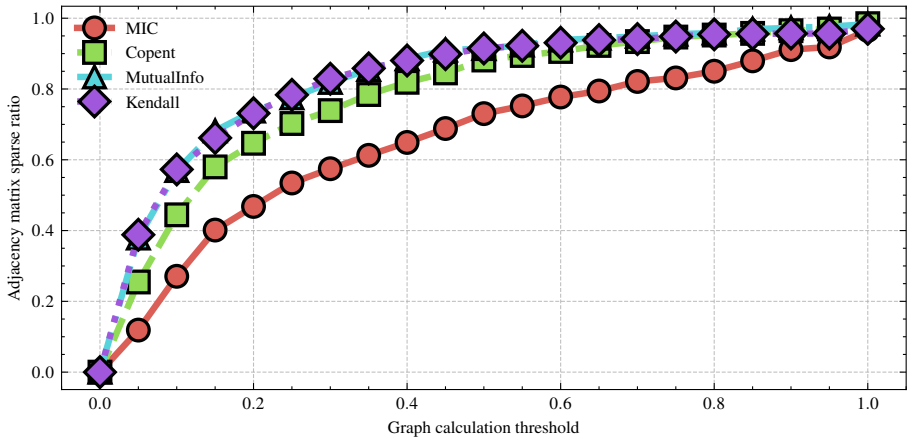

Fig. E.14. Qualitative comparison of different nonlinear correlation measures on the UNCC Dataset.

Copent, MIC achieved lower sparsity across various thresholds, indicating a greater capacity for identifying underlying nonlinear dependencies.

Appendix F. Discussion on the time cost of MIC calculation

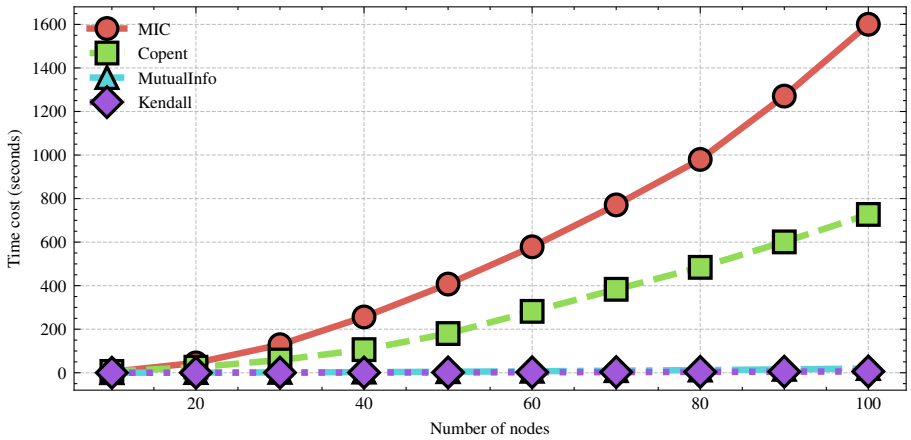

Fig. F.15. Time cost of MIC calculation with different number of nodes.

Furthermore, we compared the computation time of various nonlinear correlation measures using the UNCC Dataset mentioned in Section Appendix E. The results are presented in Figure F.15. It can be observed that among MIC, Kendall coefficient, and Copent, the measures that achieve better performance require longer

computation time, which implies that researchers need to balance accuracy and computational efficiency. In our work, since a static graph is adopted, MIC only needs to be computed once and can be reused throughout the entire process, making its computational cost acceptable. Nevertheless, if future researchers require faster computation, two alternative strategies may be considered: (1) Trade-off: one option is to adopt other measures such as Kendall coefficient or Copent, which involve shorter computation time but may sacrifice some performance. (2) Neuralization: another alternative is to neuralize the MIC computation process, an inspiring reference is the work of Matthew et al. [4], which demonstrates that neural networks can approximate the computation of mutual information.

Appendix G. Recommendations for threshold selection in graph construction

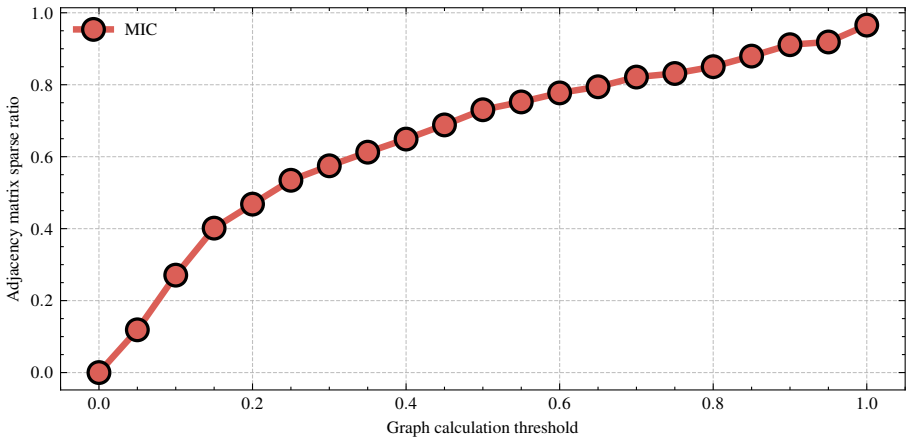

Fig. G.16. The sparsity of the adjacency matrix with different thresholds.

In fact, the issue of threshold selection is a long-standing challenge, especially when constructing graph structures based on statistical methods. Although this challenge exists not only when using MIC but also when employing other statistical methods such as cosine similarity, we can provide some suggestions for future researchers.

First, in terms of quantitative analysis, we present an experiment result showing how the sparsity of the

[4]* Matthew O'Shaughnessy, Gregory Canal, Marissa Connor, Mark Davenport, and Christopher Rozell. 2020. Generative causal explanations of black-box classifiers. In Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 458, 5453-5467.

adjacency matrix changes with different thresholds, as illustrated in Figure G.16. As described in Section Appendix E, this experiment was conducted on UNCC Dataset, a composite dataset concatenated from multiple sources and containing diverse types of nonlinear correlations. Hence, the results have a certain degree of reference value for threshold selection. Concretely, an MIC value of 0-0.2 typically indicates no or negligible correlation, 0.2-0.35 indicates potential weak nonlinear correlation requiring further validation, 0.35-0.4 suggests moderate to strong correlation that is usually statistically significant, and values above 0.5 can generally be regarded as strong correlations in most scenarios.

Second, in terms of qualitative analysis, some debugging experiences are as follows: (1) In most cases, after determining a rough range as described above, further fine-tuning is needed like other hyperparameters of neural networks, but it does not need to be fine-tuned to two decimal places, one decimal place is sufficient. (2) Relative thresholds are more reliable, the optimal threshold is often highly dependent on the specific characteristics of the dataset (such as the density of relevant information, the true correlation patterns between variables, data distribution, etc.). For example, in datasets with high sensor correlation density, modeling may not require inclusion of all correlation information, and a relatively higher threshold can be used. Conversely, when correlations are sparse, a lower threshold may be necessary to retain sufficient sensor information. Therefore, we recommend adopting relative thresholds rather than absolute ones when constructing graphs. For instance, retain the top 50% of MIC-ranked edges or the top-k strongest edges, instead of applying a universal absolute cutoff across different datasets.

Appendix H. Existing researches on graph heterophily

In fact, there have been some studies [5] [6] [7] [8] on graph heterophily in the field of GNN, but regrettably, they are not suitable for the MTS tasks in CCMS. Specifically, as shown in Table H.6, the features of

[5]* C. Zhang, D. Song, C. Huang, A. Swami, N. V. Chawla, Heterogeneous graph neural network, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 793-803. doi:10.1145/3292500.3330961. URL https://doi.org/10.1145/3292500.3330961

[6]* H. Kim, J. Choi, J. J. Whang, Dynamic relation-attentive graph neural networks for fraud detection, in: 2023 IEEE International Conference on Data Mining Workshops (ICDMW), 2023, pp. 1092-1096. doi:10.1109/ICDMW60847.2023.00143.

[7]* H. Chen, Y. Lu, Y. Yang, Y. Rao, A drug combination prediction framework based on graph convolutional network and heterogeneous information, IEEE/ACM Transactions on Computational Biology and Bioinformatics 20 (3) (2023) 1917-1925. doi:10.1109/TCBB.2022.3224734.

[8]* H. Zhang, X. An, Q. He, Y. Yao, F. Fan, Y. Teng, Quadratic graph attention network (Q-GAT) for robust construction of gene regulatory networks, CoRR abs/2303.14193 (2023). arXiv:2303.14193, doi:10.48550/ARXIV.2303.14193. URL https://doi.org/10.48550/arXiv.2303.14193

Table H.6: Existing researches on graph heterophily

Terms	Existing researches on graph heterophily	Our work
tasks scenario	citation network, social network, fraud detection, drug discovery, and gene regulation	multi-sensor time series
tasks	node classification, link prediction, etc.	multi-sensor time series reconstruction and forecasting
node feature	discrete encoded labels by hand, such as user profile, item attributes, etc.	continuous time series
edge feature	discrete encoded labels by hand, such as user-item interactions, article citations, etc.	physical law functions f(x)
heterophily connotation	semantic category differences	physical interaction nonlinearities
solution strategies	distinguish different types of nodes/edges	model the physical interaction function f(x)
heterophily processing methods	(1) simplifying heterogeneous graphs into homogeneous graphs, identifying more suitable neighbors; (2) meta-path design path patterns, such as "paper-author-paper", "paper-conference-paper"; (3) relation-specific transformation, distinctly modeling "citation edges" and "collaboration edges".	approximating the physical law function f(x) by GNN

these researches are different from ours. Therefore, because of the differences in node types, edge types, feature form, and task target, the solution strategies [9] [10] [11] employed in these scenarios, cannot be effectively transferred to MTS tasks in CCMS scenarios. Thus, it is necessary to explore new GNNs to address the issue of CCMS not conforming to the homophily assumption.

In addition to the mainstream studies mentioned above, we also note a few recent works that focus on graph heterophily in time series. In fact, these studies still belong to the expansion of the transportation field [12] , and the heterophily they study is actually local traffic flow heterophily. For example, road maintenance at certain intersections may lead to dissimilar flow patterns at nearby nodes, or the entrance of a highway

[9]* S. Wang, G. Cao, W. Cao, Y. Li, Nla-gnn: Non-local information aggregated graph neural network for heterogeneous graph embedding, Pattern Recognition 158 (2025) 110940. doi:https://doi.org/10.1016/j.patcog.2024.110940. URL https://www.sciencedirect.com/science/article/pii/S0031320324006915

[10]* T. Wang, D. Jin, R. Wang, D. He, Y. Huang, Powerful graph convolutional networks with adaptive propagation mechanism for homophily and heterophily, Proceedings of the AAAI Conference on Artificial Intelligence 36 (4) (2022) 4210-4218. doi:10.1609/aaai.v36i4.20340. URL https://ojs.aaai.org/index.php/AAAI/article/view/20340

[11]* X. Li, R. Zhu, Y. Cheng, C. Shan, S. Luo, D. Li, W. Qian, Finding global homophily in graph neural networks when meeting heterophily, in: ICML 2022, 2022. URL https://www.microsoft.com/en-us/research/publication/finding-global-homophily-in-graph-neural-networks-when-meeting-heterophily/

[12]* S. Guo, Y. Lin, H. Wan, X. Li, G. Cong, Learning dynamics and heterogeneity of spatial-temporal graph data for traffic forecasting, IEEE Transactions on Knowledge and Data Engineering 34 (11) (2022) 5415-5428. doi:10.1109/TKDE.2021.3056502.

may show different flow behaviors compared to adjacent ordinary roads. However, as mentioned in Section 3.2, even if there are heterophilic special cases in some local areas, from the global perspective, the global urban traffic sensor network is still homophilic. Some architectural improvements and module combinations can solve these local heterophily issues without replacing GCN or GAT, while still requiring the homophilic GNNs to capture and utilize the global homophily. These approaches, however, prove inapplicable to CCMS MTS tasks. These traffic studies reflect "not all homophily" scenarios, whereas our work addresses "all not homophily" conditions. Thus, our research is positioned as GNN investigation for global heterophily of MTS tasks. Although also categorized as temporal-heterophilic GNNs, it constitutes a distinct branch from the aforementioned studies.

Appendix I. Spatial Correlation Mining

In the field of MTS analysis, a well-developed framework for temporal correlation mining has been established, encompassing numerous studies and applications of CNN-based, RNN-based, and Transformer-based methods. In contrast, spatial correlation mining methods are still under exploration. Early research predominantly utilized CNN-based approaches and attention-based methods to capture spatial correlations. In recent years, with researchers recognizing the significant advantages of GNNs in modeling non-Euclidean characteristics, many GNN-based spatial correlation mining methods have been proposed, achieving promising results in some SCMS application scenarios. However, constrained by the homophily assumption, these GNN-based spatial correlation mining methods face bottlenecks when further applied to CCMS application scenarios, leading some researchers to question their validity. [13] Nevertheless, the path of graph neural networks is sound. CNN based and attention-based methods essentially perform full-range weighting, which fail to adequately capture the inherent non-Euclidean structured characteristics. To illustrate some advantages of GNNs, consider a simple example involving a physical system with N sensors, where sensor 1 is related to sensor 2 and 3 but is independent of other sensors. When using CNN-based and attention-based methods to capture the correlation of sensor 1, data from all sensors will be involved in the computation, resulting in other sensors, aside from sensor 2 and 3, acting as noise that hinders the reconstruction or prediction of sensor 1. Additionally, CNNs or attention require setting the weights of unrelated sensors to zero, which

[13]* Z. Shao, F. Wang, Y. Xu, W. Wei, C. Yu, Z. Zhang, D. Yao, G. Jin, X. Cao, G. Cong, C. S. Jensen, X. Cheng, Exploring progress in multivariate time series forecasting: Comprehensive benchmarking and heterogeneity analysis (2023). arXiv:2310.06119.

complicates the training process. In contrast, while using the GNN-based method, guided by the adjacency matrix, the correlation capture of sensor 1 will only use the data from sensor 2 and sensor 3, thus having less computational resource consumption, less noise introduction, and being easier to train. Therefore, despite encountering bottlenecks, GNN-based methods should not be abandoned, we advocate for the exploration of new GNN approaches suitable for MTS tasks in CCMS.

Appendix J. Numerical Experiments Task Description

Time series reconstruction is often employed as a preliminary step for anomaly detection, denoising, or missing data imputation. In this work, we adopt the reconstruction task rather than forecasting because the focus is on evaluating the ability of GNNs to capture spatial dependencies, rather than forecasting temporal patterns such as trends or periodicity. Our experimental approach aligns closely with standard practices in the field, with the primary difference being that in reconstruction tasks, the output corresponds to the same time steps as the input, whereas in forecasting tasks, the output pertains to future time steps. Specifically, during the data reconstruction experiments, we disable skip connections and self-connection strategies, and completely shuffle the temporal order of the dataset. That is, for each sensor node, it cannot see its own information but can only access information from other sensor nodes, and it cannot capture temporal information such as periodicity and trends. So the GNNs are forced to utilize information from other sensors to reconstruct the time series of the current sensor, allowing for an absolutely objective evaluation of the model's ability to capture nonlinear correlations between sensors. Its formulation is as follows:

Let $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T]^\top \in \mathbb{R}^{T\times D}$ be a multivariate time series with T time steps and D sensors (or nodes).

If the missing data reconstruction task is considered, some data in $\mathbf{X}$ are missing due to sensor failures or transmission loss. We define a binary mask $\mathbf{M} \in \{0, 1\}^{T\times D}$ such that

$$M_{t,d} = \begin{cases} 1, & \text{if } X_{t,d} \text{ is observed,} \\ 0, & \text{if } X_{t,d} \text{ is missing.} \end{cases} \tag{J.1}$$

The observed (incomplete) series is denoted as $\mathbf{X}_{\text{obs}} = \mathbf{M} \odot \mathbf{X}$, where $\odot$ is the element-wise product.

In this work, we assume that the D sensor graph $\mathcal{G} = (\mathbf{X}, \mathbf{A})$, where $\mathbf{A} \in \mathbb{R}^{D\times D}$ is the adjacency matrix encoding pairwise correlations among sensors. To force the model to rely solely on inter-sensor correlations

for reconstruction, we disable skip connections and self-connection strategies:

$$A_{i,j} = \begin{cases} 1, & \text{if } i \neq j \text{ and sensors } i \text{ and } j \text{ are correlated,} \\ 0, & \text{if } i = j \text{ or sensors } i \text{ and } j \text{ are not correlated.} \end{cases} \tag{J.2}$$

This constraint ensures that each node cannot access its own features directly, and GNNs must reconstruct its signal purely by aggregating information from other nodes.

The reconstruction model is defined as a graph-based mapping function $\mathcal{F}_\theta : \mathbb{R}^{T\times D} \times \mathbb{R}^{D\times D} \to \mathbb{R}^{T\times D}$, parameterized by θ, which produces the reconstructed series:

$$\widehat{\mathbf{X}} = \mathcal{F}_\theta(\mathbf{X}_{\text{obs}}, \mathbf{A}), \tag{J.3}$$

where $\mathcal{F}_\theta$ is implemented using GNNs with no skip connections, and $A_{ii} = 0, \forall i \in \{1, \ldots, D\}$.

The optimization objective focuses on:

$$\min_\theta \ \mathcal{L}_{\text{rec}} = \|\widehat{X}_{t,d} - X_{t,d}\|_p^p. \tag{J.4}$$

where $\|\cdot\|_p$ is typically the L_1 or L_2 norm, we deployed the L_2 norm in this work.

For evaluation, we measure the reconstruction quality using MSE and MAE:

$$\text{MAE} = \frac{1}{TD}\sum_{t=1}^{T}\sum_{d=1}^{D}|\widehat{X}_{t,d} - X_{t,d}|, \quad \text{MSE} = \frac{1}{TD}\sum_{t=1}^{T}\sum_{d=1}^{D}(\widehat{X}_{t,d} - X_{t,d})^2. \tag{J.5}$$